



NATIONAL SENIOR CERTIFICATE EXAMINATION
MAY 2026

INFORMATION TECHNOLOGY: PAPER I

Time: 3 hours

150 marks

PLEASE READ THE FOLLOWING INSTRUCTIONS CAREFULLY

1. This question paper consists of 16 pages. Please check that your question paper is complete.
2. This question paper is to be answered using object-oriented programming principles. Your program must make sensible use of methods and parameters.
3. This paper is divided into two sections. All candidates must answer both sections.
4. This paper is set in programming terms that are not specific to any particular programming language (Java/Delphi) or database (Access / MySQL / JavaDB).
5. Ensure that you answer the questions in the manner described because marks will be awarded for your solution according to the specifications that are given in the question.
6. Only answer what is asked for in each question. For example, if the question does not ask for data validation, no marks are awarded for it; therefore, no code needs to be written for data validation.
7. If you cannot get a section of code to work, comment it out so that it will not be executed, and you can continue with the examination. If possible, try to explain the error to aid the marker.
8. Your programs must be coded so they will work with any data, not just the sample data supplied or any data extracts that appear in the question paper. You are advised to look at the provided data files carefully.
9. Make sure that routines such as searches and sorts for arrays are developed from first principles and that you do not use the built-in features of a programming language for any of these routines.
10. All data structures must be defined and declared by you, the programmer. You may not use the interface's components to store and retrieve data.

11. Read the whole question paper before you choose a data structure. You may find that there could be an alternative method of representing the data that will be more efficient considering the questions asked in the paper.
12. You must save all your work regularly on the disk that you have been given or the disk space allocated to you for this examination. You should also create a backup of the original files before you start in case your solution accidentally modifies the original version.
13. If your examination is interrupted by a technical problem such as a power failure, you will, when you resume writing, be given only the time remaining when the interruption began to complete your examination. No extra time will be given to catch up on work that was not saved.
14. Make sure that your examination number appears as a comment in every program that you code and on every printed page that you hand in.
15. Check every printed page that you print, making sure that you submit the code for every question.
16. Check that your printout has correct indenting and formatting and has not truncated any text.
17. Check that each page is clearly legible with a font size of 12.
18. Print a code listing of all the programs/class files that you code. Printing must be done after the examination. You will be given half an hour to print after the examination. Your teacher will tell you what arrangements have been made to print your work.
19. You should have the following two folders (in bold) and files. These files are to be used as data for this examination. Note that the database files are in MS Access, JavaDB and MySQL formats and contain the same data in each file. You only need the file that matches the database application that you will need for Section A. Ensure that you can open the files with the packages that you will use to code your solutions to this examination.

Section A:

StationerySA_Access.mdb
StationerySA_JavaDB.sql
StationerySA_MySQL.sql
SQLAnswerSheet.rtf
SQLBrowser.exe

Section B:

customers.txt
orders.txt

SCENARIO

Stationery SA sells stationery items. They have a number of suppliers who help them keep stock of the items that they sell. Some items may be supplied by more than one supplier, and an individual supplier may sell many items.

SECTION A SQL**QUESTION 1**

A database called StationerySA has been developed. The database contains three tables called tblItems, tblSuppliers and tblItemSuppliers. This database holds the records of the items that StationerySA stocks, the suppliers of the items and which specific items each supplier provides.

tblItems contains the details of the items.

FIELDS	DATA TYPE	DESCRIPTION
ItemID	INTEGER	A unique auto-numbered identification number for the item.
ItemName	TEXT	The name of the stationery item.
Category	TEXT	The category of the item. This will be one of Paper, Writing Supplies, Adhesives, Cutting Tools or Fasteners.
CurrQty	INTEGER	The current quantity of the item in stock.
LowQty	INTEGER	The lowest quantity that a stock item may reach before more stock needs to be ordered.
LastOrder	DATE	The date when this item was last ordered from the supplier.

The first 8 records of tblItems:

ItemID	ItemName	Category	CurrQty	LowQty	LastOrder
1	A4 Printer Paper (500 sheets)	Paper	82	24	2024/09/07
2	A5 Spiral Notebook	Paper	59	20	2025/07/12
3	Manila Envelopes (C4, pack of 50)	Paper	21	30	2024/04/05
4	Sticky Notes (Pack of 5)	Adhesives	43	32	2025/07/28
5	Glue Stick (21g)	Adhesives	78	37	2025/03/05
6	Clear Tape (18mm × 50m)	Adhesives	32	38	2024/06/12
7	HB Pencils (Box of 12)	Writing Supplies	86	39	2024/08/08
8	Ballpoint Pens (Blue, pack of 10)	Writing Supplies	86	22	2025/01/24

tblSuppliers contains the details of the suppliers.

FIELDS	DATA TYPE	DESCRIPTION
SupplD	INTEGER	A unique auto-numbered identification number for the supplier.
SuppName	TEXT	The name of the supplier.
Contact	TEXT	The email address of the supplier.
NumOrders	INTEGER	The number of orders that Stationery SA has placed with this supplier.
Preferred	BOOLEAN	This stores whether the supplier is preferred or not.

The first 5 records of tblSuppliers:

SupplD	SuppName	Contact	NumOrders	Preferred
1	Cape Stationers	orders@capestationers.co.za	39	No
2	Joburg Office Depot	sales@joburgofficedepot.co.za	32	Yes
3	Durban Supplies Co.	hello@durbandsupplies.co.za	32	No
4	Pretoria Paper & Pen	service@pretoriapen.co.za	55	Yes
5	Gqeberha OfficeMart	support@gqeberhaofficemart.co.za	23	No

tblItemSuppliers contains the details of which items are supplied by which supplier.

FIELDS	DATA TYPE	DESCRIPTION
ItemID	INTEGER	The ID of the item. This is a foreign key to tblItems.
SupplD	INTEGER	The ID of the supplier. This is a foreign key to tblSuppliers.
ItemPrice	REAL	The price that the supplier charges for the item.
LeadTime	INTEGER	The number of days that it will take for the item to arrive.

The first 10 records of tblItemSuppliers:

ItemID	SupplD	ItemPrice	LeadTime
1	1	52.54	7
1	3	83.60	10
1	5	69.22	4
2	3	70.34	1
2	4	95.93	10
3	1	94.43	6
3	4	79.97	4
4	3	72.67	2
5	2	88.87	4
6	1	69.51	9

Write SQL statements to answer questions 1.1 to 1.10.

Note: Do not rename columns/use an alias unless explicitly told to do so in a question.

- 1.1 Display all the details of the items with a current quantity greater than 50 that were last ordered in 2024. The correct output is shown below.

ItemID	ItemName	Category	CurrQty	LowQty	LastOrder
1	A4 Printer Paper (500 sheets)	Paper	82	24	2024/09/07
7	HB Pencils (Box of 12)	Writing Supplies	86	39	2024/08/08
9	Highlighters (Pack of 4)	Writing Supplies	64	28	2024/09/27
12	Metal Scissors (210mm)	Cutting Tools	93	43	2024/02/16
13	Utility Knife	Cutting Tools	97	49	2024/03/22
17	Binder Clips (Assorted, 24pc)	Fasteners	71	22	2024/06/16

(6)

- 1.2 Display the item name and category for each item that comes in a pack and is not a writing supplies item. Items that come in a pack have the word 'pack' somewhere in their item names. The correct output is shown below.

ItemName	Category
Manila Envelopes (C4, pack of 50)	Paper
Sticky Notes (Pack of 5)	Adhesives
Paper Clips (1 000 pack)	Fasteners

(5)

- 1.3 Display the total number of orders that have been placed with all suppliers. The correct output is shown below.

TotalOrders
181

(3)

- 1.4 Display how many items are sold in each category and the category. Label the column with the number of items "NumTypes". Sort the list from the most to the least number of types. The correct output is shown below.

NumTypes	Category
7	Writing Supplies
4	Fasteners
3	Paper
3	Cutting Tools
3	Adhesives

(5)

- 1.5 Display the item name and the item price of all items with a lead time of less than 6 days. The first 6 rows of output are shown below.

ItemName	ItemPrice
A4 Printer Paper (500 sheets)	69.22
A5 Spiral Notebook	70.34
Manila Envelopes (C4, pack of 50)	79.97
Sticky Notes (Pack of 5)	72.67
Glue Stick (21g)	88.87
Clear Tape (18 mm × 50m)	74.80

(6)

- 1.6 Display the item name and current quantity of all items with quantities greater than the average quantity. The first 6 rows of output are shown below.

ItemName	CurrQty
A4 Printer Paper (500 sheets)	82
Glue Stick (21g)	78
HB Pencils (Box of 12)	86
Ballpoint Pens (Blue, pack of 10)	86
Highlighters (Pack of 4)	64
Erasers (Pack of 4)	91

(5)

- 1.7 Display the item name, supplier name and contact for all items where the current quantity of items is less than the low quantity warning level. Only show rows if the supplier is preferred. The correct output is shown below.

ItemName	SuppName	Contact
Manila Envelopes (C4, pack of 50)	Pretoria Paper & Pen	service@pretoriapen.co.za
Clear Tape (18mm x 50m)	Joburg Office Depot	sales@joburgofficedepot.co.za
Stapler (Desktop)	Pretoria Paper & Pen	service@pretoriapen.co.za
Staples (Box of 5 000)	Joburg Office Depot	sales@joburgofficedepot.co.za
Staples (Box of 5 000)	Pretoria Paper & Pen	service@pretoriapen.co.za

(5)

- 1.8 Display a list of all items that are not linked to a supplier in tblItemSuppliers. The correct output is shown below.

ItemName
Paper Clips (1 000 pack)

(4)

- 1.9 Gqeberha OfficeMart has decided to start supplying adhesives. Insert a row into tblItemSuppliers for every item that is in the adhesive category. The item ID must remain the same as the original, and the supplier ID can be hardcoded to 5. The item price must be 10% cheaper than the original item price, and the lead time can be set to 6. (7)
- 1.10 Stationery SA has decided to rename the category 'Cutting Tools' to 'Slicers'. Write a query to do so. (4)

50 marks

SECTION B OBJECT-ORIENTED PROGRAMMING

Stationery SA has regular customers as well as business customers. A business customer is another business that purchases stationery from Stationery SA. Some customers are eligible to receive a discount when making a purchase. The discount is a flat R200 off of any purchase with a value of over R1 000.

Business customers will have a length of time, called the payment term, in which to pay for their purchase. Stationery SA offers the following payment terms, which will be represented by a constant value, as shown below:

- **DAYS30 = 0** – The business has 30 days to pay off their purchase.
- **DAYS60 = 1** – The business has 60 days to pay off their purchase.
- **DAYS90 = 2** – The business has 90 days to pay off their purchase.

Customer Class

This class will represent the following basic details of a regular customer:

- **name** – The full name of the customer, stored as a string.
- **regDate** – The date that the customer registered their details, stored as a Date.
- **discount** – A boolean value representing whether the customer is eligible for a discount.

BusinessCust Class

This class will represent the basic details of a business customer with the following fields:

- **terms** – The payment terms of the business, stored as an integer.

The details of the customers and business customers are stored in the text file **customers.txt** in no specific order.

A line of the file representing a customer account has the following format:

<name>#<regDate>#<discount>

For example:

Mandisa Ndlovu# 13 08 2021#false

regDate is a date in the format dd<space>MM<space>yyyy

A line of the file representing a business customer has the following format:

<name>#<regDate>#<discount>#<payTerms>

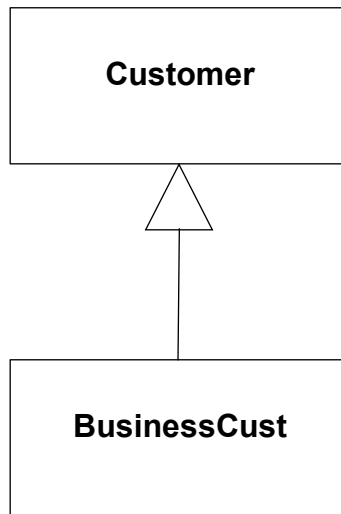
For example:

BrightPath Media#17 10 2023#true#2

The first 5 lines of data from the **customers.txt** text file are shown below.

```
Mandisa Ndlovu#13 08 2021#false
BrightPath Media#17 10 2023#true#2
Jabulani Mthembu#03 09 2020#true
Palesa Radebe#11 12 2022#false
CrimsonCloud IT#27 10 2023#false#1
```

The relationship between the parent class **Customer** and the child class **BusinessCust** is shown in the diagram below.



QUESTION 2

Use the class diagram below to create a new class called **Customer**. This class will be used to store the details of regular customers. The diagram below indicates the fields and methods that are required.

Customer
Fields: - name : string - regDate : date - discount : boolean
Methods: + Constructor(inN : string, inRD : date, inD : boolean) + getRegDate() : date + getDiscount() : boolean + getName() : string + setDiscount(inD : boolean) + toString() : string

- 2.1 Create a new class named **Customer** with the **name**, **regDate** and **discount** fields as shown in the class diagram. These fields must not be accessible outside of the class. (3)
- 2.2 Code a parameterised constructor method that will accept parameters as shown in the class diagram and assign these values to the corresponding fields. **inN** corresponds to **name**, **inRD** corresponds to **regDate** and **inD** corresponds to **discount**. (3)
- 2.3 Code accessor methods for the **regDate**, **discount** and **name** fields as shown in the class diagram. (2)
- 2.4 Code a mutator method for the **discount** field as shown in the class diagram. (2)
- 2.5 Code a **toString** method that will return a string that combines the values of the **name**, **regDate** and **discount** fields. The **regDate** field can be displayed using the default format of the Date object. The format of the string to be returned should be as follows:

name<tab>**regDate**<tab>**discount**

For example:

Mandisa Ndlovu 2021-08-13 false

Note: If you are coding in Delphi, your dates will display as yyyy/MM/dd, for example, 2021/08/13.

(2)
[12]

- 3.7 Code a **toString** method that will override the parent class's **toString** method, add the **getTermsString** result to the **toString** and return a string in the following format:

```
name<tab>regDate<tab>discount<tab>getTermsString()
```

For example:

```
BrightPath Media      2023-10-17      true      90 days      (3)
```

Note: If you are coding in Delphi, your dates will display as yyyy/MM/dd, for example, 2023/10/17.

[22]

QUESTION 4

- 4.1 Create a class called **CustomerManager**. (1)
- 4.2 Create two fields for the **CustomerManager** class as described below:
- An array called **cArr** to store up to 40 **Customer** or **BusinessCust** objects.
A counter called **size** to count the objects added to the array.
These fields should not be accessible from outside of the class. (3)
- 4.3 Code a constructor method that will read the contents of the text file **customers.txt**, which contains the information about the **Customer** and **BusinessCust** accounts. The method should do the following:
- Read each line from the file **customers.txt** and extract the data.
 - Instantiate either a **Customer** or **BusinessCust** object based on the line's extracted data.
 - Add the **Customer** or **BusinessCust** object to the next available position in the array.
 - Increase the counter **size**. (13)
- 4.4 Code a **toString** method to combine the values of each object in the array **cArr** into a string. Add a heading called '**Customer List**'. The details of each object should be on a new line. Use the **toString** methods that you created in Question 2 and Question 3. (4)
- 4.5 Code a method called **countDiscounts** that will return the combined number of **Customer** and **BusinessCust** objects that receive a discount. (5)
- 4.6 Code a method called **updateDiscount**. This method takes two parameters: The first is the name of the customer whose discount we want to update, and the second is the boolean value of the discount that we wish to assign to the customer in **cArr** whose name matches the name parameter. Assign the discount to the customer whose name matches the first parameter. The method should use an appropriate looping structure that stops as soon as the correct customer is found. (5)

[31]

QUESTION 5

- 5.1 Create a text-based user interface called **CustomerUI** that will allow simple input and output. Declare and instantiate a **CustomerManager** object. (1)
- 5.2 Print the contents of the **CustomerManager** object that you created in Question 5.1. The output is shown below Question 5.4. (1)
- 5.3 Print the total number of customers who are eligible for discounts. The correct answer is 8. (1)
- 5.4 Update the following discount details. **Busisiwe Dube's** discount value should be set to false. (1)

The first 6 lines of output from Question 5.2 are displayed below. Note: If you are coding in Delphi, your dates will display as yyyy/MM/dd, for example, 2021/08/13.

```
Customer List
Mandisa Ndlovu      2021-08-13    false
BrightPath Media    2023-10-17    true   90 days
Jabulani Mthembu    2020-09-03    true
Palesa Radebe       2022-12-11    false
CrimsonCloud IT     2023-10-27    false  60 days
Karabo Seabi        2021-07-21    false
```

[4]

QUESTION 6

The following questions use the text file "**orders.txt**". The text file contains a number of lines of text in the format:

name#cost

For example: **BrightPath Media#1855**

Each line in the text file shows the name of a **BusinessCust** and the amount of money that they used to purchase products. A single name may be repeated multiple times, representing multiple purchases from that same business customer. The first 10 lines of the text file are shown below.

```
BrightPath Media#1855
LionHeart Security#2564
SilverLeaf Exports#7332
CrimsonCloud IT#2094
CrimsonCloud IT#2912
LionHeart Security#7362
LionHeart Security#2282
BrightPath Media#6004
CrimsonCloud IT#9545
LionHeart Security#5223
```

In the **CustomerManager** class, do the following:

- 6.1 Code a method called **countOrdersInFile** that accepts a string parameter representing the filename of a text file and returns an integer. The method should determine the number of lines in the given text file and return the number of lines. (5)
- 6.2 Code a method called **getOrders** that accepts a string parameter representing the filename of a text file. The text file will contain a number of lines, representing purchases, as explained above. The method should return an array of strings consisting of the lines of text from the file sent through as a parameter.

Your method should work for the given text file and any other text files, even if they have a different number of lines.

- Each element of the array should contain exactly one line of text from the text file.
- The array should be exactly the same size as the number of lines in the text file.
- You may not hardcode this size when reading from the file and returning the array.

(5)

- 6.3 Code a method called **generateTotals**. Use the helper method **getOrders** to help you code **generateTotals**. This method accepts a string parameter representing the filename of a text file. The method returns a string with information as described below.

The method **generateTotals** should do the following:

- Go through the array of customers.
- For every business customer:
 - Calculate the total value of the purchases that they made. Remember that some customers are eligible for a discount, which is a flat R200 off of their total purchase for any purchase over R1 000. Determine the total amount owed after the discount (if any) is applied.
 - After determining the total amount owed by the business customer, determine on which day the amount is due. The due date is the number of days represented by the payment terms of the business. For example, if the payment term is DAYS60, then the due date is 60 days from the current date. DAYS30 gives 30 days from the current date. DAYS90 gives 90 days from the current date. Otherwise, instead of displaying the due date, the message should be "to be paid today".
 - Finally, the method should build a string in the following or similar format:

```
name<space>"owes"<space>total<space>"to be paid by"
<space>dueDate
```

For example:

BrightPath Media owes 34685 to be paid by 2026-01-08

The details of each customer should be printed on separate lines, as shown in the output after Question 6.4.

(20)

- 6.4 Return to the **CustomerUI** class. Add code to print the generated totals from Question 6.3, using the parameter **"orders.txt"**. The correct output is shown below.

Note: The sample data was generated using 30 May 2026 as the current date. Your dates will vary depending on the day that you write.

Note: If you are coding in Delphi, your dates will display as yyyy/MM/dd, for example, 2026/02/22.

```
BrightPath Media owes 34685 to be paid by 2026-08-28
CrimsonCloud IT owes 32287 to be paid by 2026-07-29
Savanna Foods owes 16470 to be paid by 2026-07-29
LionHeart Security owes 50420 to be paid by 2026-06-29
Evergreen Logistics owes 29564 to be paid by 2026-07-29
SilverLeaf Exports owes 22431 to be paid by 2026-07-29
MetroBuild Contractors owes 18816 to be paid by 2026-08-
28
```

(1)
[31]

100 marks

Total: 150 marks